第六讲 - 流程控制 (编程练习)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 业务问题引入

2 程序如何选择和重复

3 条件语句

4 for循环

5 while循环

6 循环控制

7 综合业务实践

8 总结与课后拓展

Python 程序设计基础

第6讲 流程控制 速记卡

让程序学会判断、重复、筛选业务数据！

1 为什么要学流程控制？

以电商订单处理为例：
面对一批订单，程序需要：

- ☑ 判断是否处理该订单
- ☑ 跳过已取消/已退款的订单
- ☑ 识别高价值订单
- ☑ 累计已支付金额

订单号	金额	状态
0001	299	PAID
0002	120	CANCELLED
0003	680	PAID
0004	1500	PAID
0005	88	REFUNDED
…	…	…

关键点：
条件语句负责“选择”，
循环语句负责“重复处理一批数据”。

2 顺序、选择与循环

1. **顺序：** 按代码先后执行。
2. **选择：** 根据条件决定做什么。
3. **循环：** 对一批数据重复执行。

if / for / while 语句后都有**冒号：**并通过缩进表示代码块。
缩进表示从属关系。

```
for order in orders:
    if order["status"] == "PAID":
        print(order["id"], order["amount"])
```

缩进表示从属关系，
意思是 if 的代码属于 for 循环内部。

3 条件语句 if / elif / else

if： 满足条件才执行

```
if amount >= 1000:
    print("高价订单")
```

if-else： 两种结果二选一

```
if amount >= 500:
    print("高价值订单")
else:
    print("普通订单")
```

if-elif-else： 多结业务规则

```
if amount >= 1000:
    print("重点订单")
elif amount >= 500:
    print("高价值订单")
else:
    print("普通订单")
```

小贴士： 先检查**边界值**；
条件越**嘴**从**严**格到宽松排列。

4 for 循环

当数据集合已提前知道时，使用 for 循环逐个处理数据。

遍历订单列表

```
for order in orders:
    print(order["id"], order["amount"])
```

遍历字典

```
for market, sales in market_sales.items():
    print(market, sales)
```

累加与计数

```
total_amount += order["amount"]
order_count += 1
```

循环与条件结合（只统计已支付订单）

```
for order in orders:
    if order["status"] == "PAID":
        total_amount += order["amount"]
```

关键点： 遍历 + 筛选 + 汇总。

5 while 循环

当重复次数不确定，但停止条件明确时，使用 while 循环。

示例1：库存补货

```
stock = 10
safe_stock = 50
while stock < safe_stock:
    stock += 5
```

示例2：输入校验

```
while True:
    s = input("请输入数量：")
    if s.isdigit():
        break
print("请输入数字！")
```

请输入数量：？

注意： 循环体必须让条件逐步接近 False；否则可能导致无限循环。

6 break 与 continue

break 立即结束整个循环

- 找到目标后，立刻结束循环。
- **示例：** 找到第一笔金额 >= 1000 的订单后停止。

```
for order in orders:
    if order["amount"] >= 1000:
        print("找到！", order["id"])
        break
```

continue 跳过当前这一条

- 跳过当前不符合条件的数据，继续检查下一条。
- 示例：遇到未支付订单，跳过统计。

```
for order in orders:
    if order["status"] != "PAID":
        continue
    total_amount += order["amount"]
```

记住： break 是“停下来”，continue 是“跳过去”。

7 综合业务实践：订单汇总

业务任务：
根据下面的订单数据，完成统计：
只统计已支付订单；计算已支付订单数、实收金额、未支付金额（金额 >= 1000），以及平均订单金额（避免除以 0）。

订单号	金额	状态
0001	299	PAID
0002	120	CANCELLED
0003	680	PAID
0004	1500	PAID
0005	88	REFUNDED

一起来计算吧！

```
# 初始变量（遍历前）
paid_count = 0
paid_amount = 0
high_amount = 0
for order in orders:
    if order["status"] == "PAID":
        paid_count += 1
        paid_amount += order["amount"]
        if order["amount"] >= 1000:
            high_amount += 1
average = paid_amount / paid_count if paid_count > 0 else 0
```

运行结果：

- 已支付订单数：3
- 实收金额：2479.00 元
- 未支付单数：1
- 平均订单金额：826.33 元

人工验证：
299 + 680 + 1500 = 2479

8 学习提醒 / 易错点

1. 先想清楚：什么时候判断？什么时候重复？什么时候停止？
2. 冒号后要缩进，通常 4 个空格。
3. if / elif / else 一次只会进入一个分支。
4. for 适合遍历已知数据；while 适合条件控制。
5. 统计前先设初值，如 total = 0，count = 0。
6. 平均值前先判断 count 是否为 0。
7. continue 不是结束循环，break 才是结束循环。
8. 复杂条件可先拆成布尔变量，例如 is_paid、is_high_value。

多写代码，多练习！

记住： 流程控制不是背语法，而是把业务规则写成“会判断、会重复、会停止”的程序。

业务任务：批量处理订单

```
orders = [
{"id": "0001", "amount": 299, "status": "PAID"},
{"id": "0002", "amount": 120, "status": "CANCELLED"},
{"id": "0003", "amount": 680, "status": "PAID"}
]
```

处理一条订单

- 根据订单状态决定是否处理；
- 已支付订单金额达到500元时，标记为高价值订单；
- 输出该订单的处理结果。

处理一批订单

- 对每条订单执行相同规则；
- 跳过已经取消的订单；
- 累计已支付金额和订单数量；
- 找出全部高价值订单。

本讲需要解决的核心问题

条件语句让程序根据业务规则选择操作；循环语句让程序把相同规则应用到一批数据。

顺序、选择与循环

```python
# 程序逐条取得订单。只有订单状态为PAID时，才读取金额并输出订单信息。
for order in orders: # 循环
  if order["status"] == "PAID": # 选择
    amount = order["amount"] # 顺序
    print(order["id"], amount)
```

三种执行方式

- **顺序**：程序按照代码的先后顺序执行操作；
- **选择**：程序根据条件决定是否执行某个代码块；
- **循环**：程序对一批数据重复执行同一个代码块。

冒号与代码块

`for` 和 `if` 语句以冒号结束。冒号后面缩进的语句构成对应的代码块。

缩进表示从属关系

本例中，`if` 属于 `for` 循环；读取金额和输出结果又属于 `if` 条件。Python通常使用4个空格表示一级缩进。

if：满足条件时执行

```
order = {"id": "0003", "amount": 680, "status": "PAID"}

order_id = order["id"]

if order["amount"] >= 500:
  print(f"{order_id}是高价值订单")

print("订单检查完成")
```

条件表达式

表达式 `order["amount"] >= 500` 的结果是 `True` 或 `False` 。

if的执行规则

条件为 `True` 时，程序执行缩进的代码块；条件为 `False` 时，程序跳过该代码块。

条件语句结束以后： `print("订单检查完成")` 没有缩进，因此不属于 `if` 。无论订单金额是多少，这条语句都会执行。

if-else：两种处理结果

```
status = "PAID"

if status == "PAID":
  action = "进入发货流程"
else:
  action = "暂不发货"

print(action)
```

两个分支

条件成立时执行 `if` 代码块；条件不成立时执行 `else` 代码块。

两个分支不会同时执行

一次判断只会选择一个分支。分支执行完成后，程序继续执行条件语句后面的代码。

适用的业务问题

是否发货、是否包邮、是否达到目标等只有两种结果的问题，适合使用 `if-else` 。

if-elif-else：多档业务规则

```
amount = 680

if amount >= 1000:
  order_level = "重点订单"
elif amount >= 500:
  order_level = "高价值订单"
else:
  order_level = "普通订单"

print(order_level)
# 高价值订单
```

判断顺序

程序从上到下依次检查条件。遇到第一个结果为 `True` 的条件后，执行对应代码块。

只执行一个分支，else处理剩余情况

即使后面的条件也可能成立，程序也不会继续判断。因此，多档条件的排列顺序会影响结果。前面的条件都不成立时，程序执行 `else` 代码块。本例把金额低于500元的订单归为普通订单。

条件顺序与边界测试

订单金额	应进入的分支	订单等级
`499.99`	`else`	普通订单
`500.00`	`amount >= 500`	高价值订单
`999.99`	`amount >= 500`	高价值订单
`1000.00`	`amount >= 1000`	重点订单

先检查边界值

业务规则以500元和1000元为分界线。测试时应检查分界线本身，以及略低于分界线的数据。

注意比较运算符

`amount >= 500` 包含500元；如果改成 `amount > 500`，500元订单将进入其他分支。

条件通常从严格到宽松排列

如果先判断 `amount >= 500`，金额为1200元时，程序会立即进入该分支，无法再进入 `amount >= 1000` 的分支。

for循环：逐条处理订单

```
# 接续本讲开头定义的orders
for order in orders:
  print(order["id"], order["amount"])

# 运行结果:
0001 299
0002 120
0003 680
```

for循环的执行过程

- `orders` 是需要遍历的订单列表；
- `order` 是循环变量，每次表示一条订单；
- 列表中有多少条订单，循环体就执行多少次。

循环体： `print()` 语句前面有一级缩进，因此属于循环体。程序会对列表中的每条订单执行这条语句。

for循环：遍历字典

```
market_sales = {"泰国": 128, "越南": 98, "印尼": 76}

for market, sales in market_sales.items():
  print(f"{market}市场: {sales}万元")

# 运行结果:
泰国市场: 128万元
越南市场: 98万元
印尼市场: 76万元
```

遍历键值对

`items()`提供字典中的键值对。每次循环分别把键赋值给`market`，把值赋值给`sales`。

循环变量的名称

循环变量应说明数据的业务含义。使用`market`和`sales`比使用`x`和`y`更容易理解。

循环中的累加与计数

```
total_amount = 0
order_count = 0

for order in orders:
  total_amount += order["amount"]
  order_count += 1

average_amount = total_amount / order_count

print(f"订单数量: {order_count}")
print(f"金额合计: {total_amount}元")
print(f"平均金额: {average_amount:.2f}元")
```

累计变量的使用步骤

1. 在循环开始前为累计变量设置初始值；
2. 在每次循环中更新累计变量；
3. 循环结束后使用最终结果。

这里统计的是列表中全部订单的记录金额，其中仍然包含已经取消的订单。

循环与条件的组合

```
paid_total = 0
paid_count = 0

for order in orders:
  if order["status"] == "PAID":
    paid_total += order["amount"]
    paid_count += 1

print(f"已支付订单数量: {paid_count}")
print(f"已支付金额合计: {paid_total}元")

# 运行结果:
已支付订单数量: 2
已支付金额合计: 979元
```

遍历、筛选与汇总

`for` 逐条取得订单，`if` 筛选已支付订单，累计变量保存筛选后的数量和金额。

缩进关系： `if` 属于 `for` 循环；两条累计语句又属于 `if` 代码块，因此需要两级缩进。

while循环：条件成立时重复

```python
stock = 12
safe_stock = 30

while stock < safe_stock:
  stock += 5
  print(f"当前库存: {stock}")

print("库存已达到安全水平")
```

执行过程

每次循环前，Python都会判断 `stock < safe_stock` 。条件为 `True` 时执行循环体；条件变为 `False` 时结束循环。

避免无限循环

循环体必须让条件逐步接近 `False` 。 本例通过 `stock += 5` 增加库存，否则循环会一直执行。

适用场景： 当重复次数不能提前确定，但停止条件明确时，通常使用 `while` 循环。

while True：输入有效后停止

```
while True:
  user_input = input("请输入商品数量: ")
  quantity_text = user_input.strip()
  # isdigit()适合检查由数字字符组成的非负整数文本，不能直接处理负数和小数。
  if quantity_text.isdigit():
    quantity = int(quantity_text)
    break
  print("输入无效，请输入非负整数。")

print(f"商品数量: {quantity}")
```

为什么需要重复输入以及循环如何结束

用户输入可能不符合要求。程序需要持续接收和检查输入，直到获得可以使用的商品数量。`while True` 创建持续循环。只有输入通过检查后，程序才执行 `break` 并结束当前循环。

break：找到结果后结束循环

```
order_amounts = [299, 1500, 680, 2300]

for amount in order_amounts:
  if amount >= 1000:
    print(f"发现第一笔大额订单：{amount}元")
    break
```

执行结果

程序找到金额为 1500 元的订单后执行 break，并立即结束当前循环。因此，后面的订单不再接受检查。

适用场景

已经找到目标结果，或者满足某个条件后不需要继续循环时，可以使用 break 减少不必要的处理。如果任务要求找出所有大额订单，就不能在找到第一笔后使用 break。

continue：跳过本轮处理

```
orders = [
{"id": "0001", "amount": 299, "status": "PAID"},
{"id": "0002", "amount": 120, "status": "CANCELLED"},
{"id": "0003", "amount": 680, "status": "PAID"}
]

paid_amount = 0

for order in orders:
  if order["status"] != "PAID":
    continue
  paid_amount += order["amount"]

print(f"实收金额：{paid_amount}元")
```

执行过程：遇到已取消订单时，程序执行 `continue`，跳过本轮后面的金额统计，然后继续处理下一笔订单。

break与continue的区别： `break` 结束整个当前循环；`continue` 只跳过当前这一轮，循环仍会继续。写在 `continue` 之后的本轮代码不会执行，使用前需要确认这些代码确实可以被跳过。

订单汇总任务

```
orders = [
{"id": "0001", "amount": 299,  "status": "PAID"},
{"id": "0002", "amount": 120,  "status": "CANCELLED"},
{"id": "0003", "amount": 680,  "status": "PAID"},
{"id": "0004", "amount": 1500, "status": "PAID"},
{"id": "0005", "amount": 88,   "status": "REFUNDED"}
]
```

统计口径

只有状态为 `"PAID"` 的订单计入统计。已取消和已退款的订单不计入实收金额。

需要回答的问题

- 已支付订单共有多少笔，实收金额是多少？
- 金额不低于1000元的大额订单有多少笔？
- 已支付订单的平均金额是多少？

编程前先明确：输入是订单列表，处理规则是筛选与累计，输出是四个汇总指标。

业务规则的程序实现

```
paid_count = 0
paid_amount = 0
high_value_count = 0

for order in orders:
  if order["status"] != "PAID":
    continue

  paid_count += 1
  paid_amount += order["amount"]

  if order["amount"] >= 1000:
    high_value_count += 1

if paid_count == 0:
  average_amount = 0
else:
  average_amount = paid_amount / paid_count
```

代码与业务规则的对应关系： `continue` 跳过未支付订单；两个累加变量分别记录订单数和实收金额；内层条件判断统计大额订单；最后的条件判断避免出现除以零的错误。

结果输出与验证

```
print(f"已支付订单数: {paid_count}")
print(f"实收金额: {paid_amount:.2f}元")
print(f"大额订单数: {high_value_count}")
print(f"平均订单金额: {average_amount:.2f}元")
```

程序输出

已支付订单数为3，实收金额为2479.00元，大额订单数为1，平均订单金额为826.33元。

人工核对

- 计入统计的订单是 `O001` 、 `O003` 和 `O004` ；
- 实收金额为$299 + 680 + 1500 = 2479$元；
- 平均金额为$2479 \div 3 \approx 826.33$元。

边界测试

把所有订单状态改为 `"CANCELLED"` 。程序应输出0笔订单、0元实收金额和0元平均金额，并且不产生错误。

流程控制的选择方法

业务问题	Python结构	选择依据
按规则执行不同操作	`if` 、 `elif` 、 `else`	程序需要根据条件进行选择
依次处理一组数据	`for`	已知需要遍历的列表、字典或其他对象
满足条件时持续执行	`while`	重复次数不能提前确定，但停止条件明确
找到结果后立即停止	`break`	继续循环已经没有必要
跳过当前记录	`continue`	当前记录不需要处理，但仍需检查后续记录

共同原则

先明确每段代码何时执行、何时结束，再用缩进表示代码的作用范围。复杂任务可以先分解为多个简单判断。

课后实践：订单筛查报告

实践任务

沿用综合实践中的 `orders` 数据。程序接收一个金额阈值，并生成已支付订单筛查报告。

功能要求

- 使用 `while` 检查输入，金额阈值必须是非负整数；
- 使用 `for` 遍历订单，跳过未支付订单；
- 输出金额不低于阈值的订单编号和金额；
- 输出符合条件的订单数量与金额合计。

验收测试

- 输入 `500` 时，应找到 `O003` 和 `O004`，合计2180元；
- 输入 `2000` 时，订单数和金额合计都应为0；
- 输入 `abc` 时，程序应提示错误并要求重新输入。

课后拓展：短路运算与复杂条件

```
coupon_count = 0
discount_total = 0

if coupon_count > 0 and discount_total / coupon_count > 50:
  print("需要检查优惠成本")
```

短路运算

当 and 左侧条件为 False 时，Python不再计算右侧条件。因此，本例不会执行除法，也不会产生除以零的错误。

```
is_paid = order["status"] == "PAID"
is_high_value = order["amount"] >= 1000

if is_paid and is_high_value:
  print("大额已支付订单")
```

复杂条件的处理方法：把复杂条件拆分为含义清晰的布尔变量，可以分别检查每个判断。

THE END